Yeshwantrao Chavan College of Engineering, Nagpur

Department of Electronics Engineering

Session2025-26

23EE1605

EDA Laboratory Report

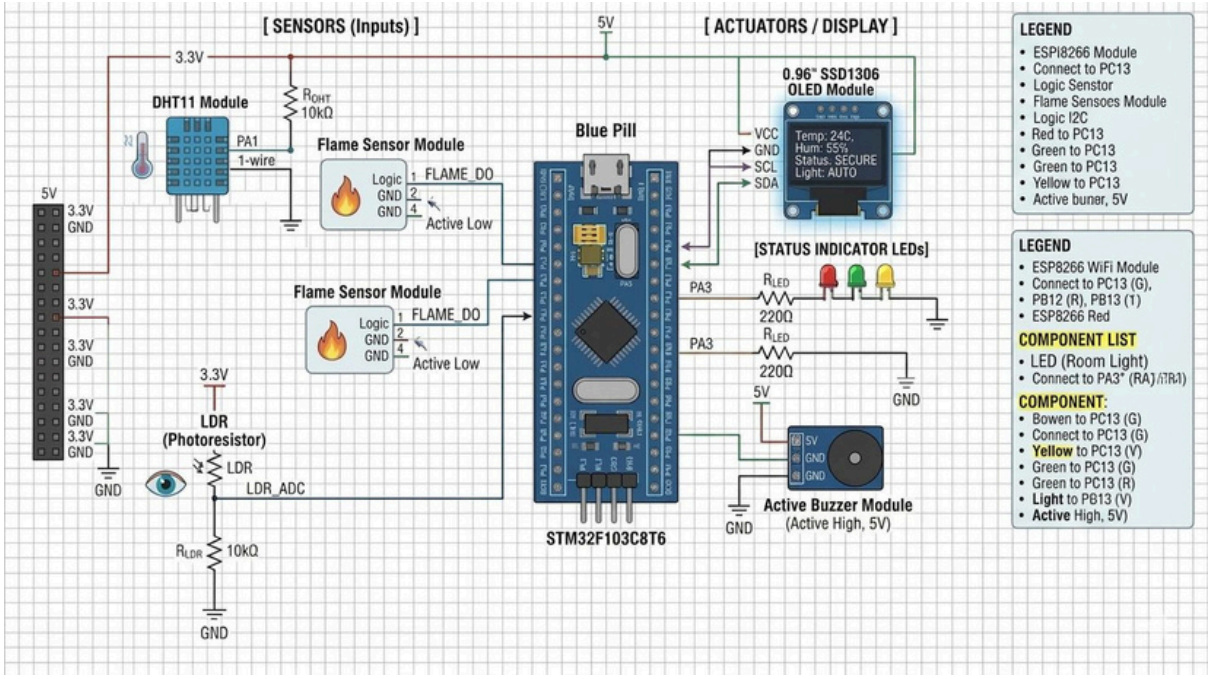
Project Title:

SMART ROOM MONITORING SYSTEM

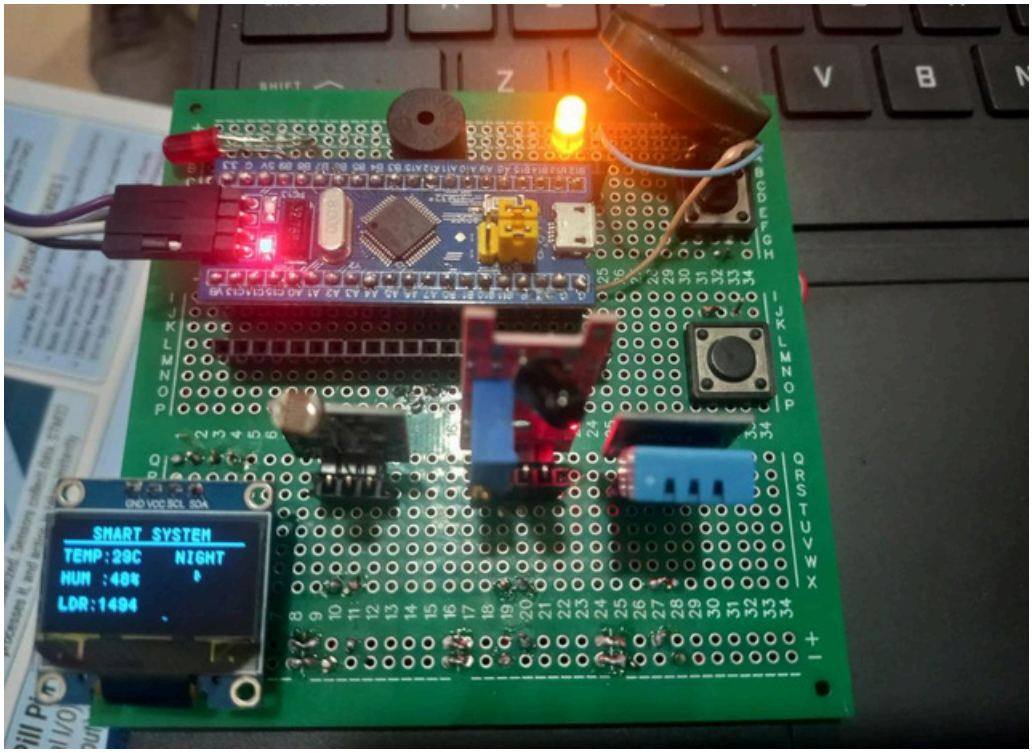
Name:	**JoyeetaDhar**	
Roll No. 12	**Batch:A1**	**Sem: 6th**
Registration No.	**23070406**	
Name of Faculty	**Dr.Pradnya Zode**	

MSPA-04

Circuit Diagram



Photographs of projects



Ports used

Port Name	Description of component attached
PA1	**DHT11 Module** (AnDigital temperature sensor that outputsvoltage proportional to temperature.
PA0	**LDR (Photoresistor)**(Measures ambient light levels. It is configured involtagedivider with10k resistor.).
PA2	**Flame Sensor Modules**(Infra-Red sensor used for fire detection.).
PA3	**LEDs** (A device that turns on when ldr senses darkness in room)
PA8	**Buzzer** (An audio output device used to generate sound).
PB6 → SCL **PB7 → SDA**	**OLED Display** (SSD1306 - I2C) Asmall display module used to show text and sensor data. It communicates with the microcontroller using the I2C protocol

Code with comments

```
/* USER CODE BEGIN Header */
/**
*****************************************************************************
  * @file           : main.c
  * @brief          : Fire Security & Auto-Lighting System
*****************************************************************************
  */
/* USER CODE END Header */

/*  Includes  ------------------------------------------------------------*/
#include "main.h"
#include "ssd1306.h"
#include "fonts.h"
#include "dht11.h"
#include <stdio.h>
```

```
/* Private variables ---------------------------------------------------------*/
ADC_HandleTypeDef hadc1;
I2C_HandleTypeDef hi2c1;
TIM_HandleTypeDef htim1;

uint16_t  ldr_value  =  0;
uint8_t  temperature  =  0;
uint8_t   humidity   =   0;
char oled_buffer[32];

/* Function Prototypes -------------------------------------------------------*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_I2C1_Init(void);
static void MX_ADC1_Init(void);
static void MX_TIM1_Init(void);

intmain(void)
{
  /*MCU Configuration */
  HAL_Init();
  SystemClock_Config();

  /*Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_I2C1_Init();
  MX_ADC1_Init();
  MX_TIM1_Init();
```

```
/*USER CODE BEGIN 2 */
HAL_TIM_Base_Start(&htim1); // Timer for DHT11 delays
HAL_Delay(500);

ssd1306_Init(&hi2c1);
ssd1306_Fill(Black);
ssd1306_SetCursor(10, 20);
ssd1306_WriteString("SYSTEM ONLINE", Font_7x10, White);
ssd1306_UpdateScreen(&hi2c1);
HAL_Delay(1000);
/*USER CODE END 2 */

/*Infinite loop */
while (1)
{
  /*1. SENSOR READING */
  DHT11_Read(&temperature, &humidity);

  HAL_ADC_Start(&hadc1);
  if(HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK) {
     ldr_value = HAL_ADC_GetValue(&hadc1);
  }
  HAL_ADC_Stop(&hadc1);

  //Read Flame Sensor (PA2)
  //Logic: RESET (0) means Fire detected, SET (1) means No Fire
  GPIO_PinState fire_detected = HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_2);

  /*2. HARDWARE CONTROL LOGIC */
```

```
//LED Control (PA3) based on LDR
//Threshold adjusted to 2000. If LED doesn't turn on, increase this number.
/*--- LDR & LED DISPLAY LOGIC --- */
if(ldr_value > 1200) {
   // It is DARK
   HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, GPIO_PIN_SET); // Turn LED ON
   ssd1306_SetCursor(0, 30);
   ssd1306_WriteString("LED ON: Night ", Font_7x10, White);
}else {
   // It is BRIGHT
   HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, GPIO_PIN_RESET); // Turn LED OFF
   ssd1306_SetCursor(0, 30);
   ssd1306_WriteString("LED OFF: Day ", Font_7x10, White);
}
/*------------------------------- */

//Buzzer Control (PA8) based on Flame Sensor
if(fire_detected == GPIO_PIN_SET) {
   HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_SET);   // BUZZER ON
}else {
   HAL_GPIO_WritePin(GPIOA, GPIO_PIN_8, GPIO_PIN_RESET); // BUZZER OFF
}

/*3. DISPLAY UPDATE */
ssd1306_Fill(Black);

//If your LED turns on with a torch, swap '<' to '>'
   if (ldr_value > 1200) {
```

```
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, GPIO_PIN_SET);   // Physical LED
ON
        ssd1306_SetCursor(0, 30);
        ssd1306_WriteString("LED: ON (Night)", Font_7x10, White);
      }else {
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3, GPIO_PIN_RESET); // Physical LED
OFF
        ssd1306_SetCursor(0, 30);
        ssd1306_WriteString("LED: OFF (Day)", Font_7x10, White);
      }
    //Display Temp/Hum
    sprintf(oled_buffer, "Temp: %d C", temperature);
    ssd1306_SetCursor(0, 0);
    ssd1306_WriteString(oled_buffer, Font_7x10, White);

    sprintf(oled_buffer, "Hum: %d %%", humidity);
    ssd1306_SetCursor(0, 15);
    ssd1306_WriteString(oled_buffer, Font_7x10, White);

    //Display Flame Status
    if(fire_detected == GPIO_PIN_SET) {
      ssd1306_SetCursor(0, 40);
      ssd1306_WriteString("!!! FIRE !!!", Font_11x18, White);
    }else{
      ssd1306_SetCursor(0, 50);
      ssd1306_WriteString("Status: Secure", Font_7x10, White);
    }

    ssd1306_UpdateScreen(&hi2c1);
    HAL_Delay(500); // 500ms refresh rate
```

```
 }
}

/**
  *@brief GPIO Initialization Function
  */
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};

  __HAL_RCC_GPIOA_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();

  /*Set Output Pins to Low initially */
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_3 | GPIO_PIN_8, GPIO_PIN_RESET);

  /*PA1: DHT11 (Input/Output handled by library)
    PA2: Flame Sensor (Input) */
  GPIO_InitStruct.Pin = GPIO_PIN_1 | GPIO_PIN_2;
  GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  /*PA3: LED Output
    PA8: Buzzer Output */
  GPIO_InitStruct.Pin = GPIO_PIN_3 | GPIO_PIN_8;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
```

```
 HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
}
/**
  * @brief This function is executed in case of error occurrence.
  * @retval None
  */
void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
  /* User can add his own implementation to report the HAL error return state */
  __disable_irq();
  while (1)
  {
      // You could toggle an LED here to show the system crashed
  }
  /* USER CODE END Error_Handler_Debug */
}
/* ... Keep your existing SystemClock_Config, MX_ADC1_Init, MX_I2C1_Init, and
MX_TIM1_Init ... */
void SystemClock_Config(void)
{

  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
  RCC_PeriphCLKInitTypeDef PeriphClkInit = {0};


  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
  RCC_OscInitStruct.HSEState = RCC_HSE_ON;
  RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
```

```
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
  RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL9;
  if(HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  RCC_ClkInitStruct.ClockType =
RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                        |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

  if(HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
  {
    Error_Handler();
  }

  PeriphClkInit.PeriphClockSelection = RCC_PERIPHCLK_ADC;
  PeriphClkInit.AdcClockSelection = RCC_ADCPCLK2_DIV6;
  if(HAL_RCCEx_PeriphCLKConfig(&PeriphClkInit) != HAL_OK)
  {
    Error_Handler();
  }
}
staticvoidMX_ADC1_Init(void)
```

```
{
  ADC_ChannelConfTypeDef sConfig = {0};

  hadc1.Instance = ADC1;
  hadc1.Init.ScanConvMode = ADC_SCAN_DISABLE;
  hadc1.Init.ContinuousConvMode = DISABLE;
  hadc1.Init.DiscontinuousConvMode = DISABLE;
  hadc1.Init.ExternalTrigConv = ADC_SOFTWARE_START;
  hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;
  hadc1.Init.NbrOfConversion = 1;
  if(HAL_ADC_Init(&hadc1) != HAL_OK)
  {
    Error_Handler();
  }

  sConfig.Channel = ADC_CHANNEL_0;
  sConfig.Rank = ADC_REGULAR_RANK_1;
  sConfig.SamplingTime = ADC_SAMPLETIME_1CYCLE_5;
  if(HAL_ADC_ConfigChannel(&hadc1, &sConfig) !=  HAL_OK)
  {
    Error_Handler();
  }
}
static void MX_I2C1_Init(void)
{
  hi2c1.Instance = I2C1;
  hi2c1.Init.ClockSpeed = 100000;
  hi2c1.Init.DutyCycle = I2C_DUTYCYCLE_2;
  hi2c1.Init.OwnAddress1 = 0;
```

```
 hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
  hi2c1.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
  hi2c1.Init.OwnAddress2 = 0;
  hi2c1.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
  hi2c1.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;
  if (HAL_I2C_Init(&hi2c1) != HAL_OK)
  {
    Error_Handler();
  }
}
static void MX_TIM1_Init(void)
{
  TIM_ClockConfigTypeDef sClockSourceConfig = {0};
  TIM_MasterConfigTypeDef sMasterConfig = {0};

  htim1.Instance = TIM1;
  htim1.Init.Prescaler = 71;
  htim1.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim1.Init.Period = 65535;
  htim1.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim1.Init.RepetitionCounter = 0;
  htim1.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
  if(HAL_TIM_Base_Init(&htim1) != HAL_OK)
  {
    Error_Handler();
  }
  sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
  if(HAL_TIM_ConfigClockSource(&htim1, &sClockSourceConfig) != HAL_OK)
  {
```

```
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if(HAL_TIMEx_MasterConfigSynchronization(&htim1, &sMasterConfig) !=  HAL_OK)
  {
    Error_Handler();
  }
}
```